



Instituto Técnico de Capacitación y Productividad -INTECAP-
Centro de Capacitación en Tecnologías de la Información y de
Comunicaciones -TICS-
Desarrollador Full Stack (2026.CTI.0502.00/DT.E.01030)

A large, faint, light blue watermark of the Intecap logo is centered in the background of the page.

Informe Proyecto Final “Tienda en Línea: TodoAquí”

Oscar Estuardo Ardon Castillo
Alba Gabriela Ortega

Guatemala, 08 de octubre de 2026

Índice

1. Resumen ejecutivo y planteamiento	4
1.1. Contexto y problema	4
1.2. Objetivo general	4
1.3. Solución implementada	4
1.4. Alcance funcional	4
2. Requerimientos y Product Backlog	5
2.1. Requerimientos funcionales	5
2.2. Requerimientos no funcionales	6
2.3. Validación de requerimientos	6
2.4. Mockups, referencias visuales y versión de interfaz	6
3. Arquitectura y tecnologías	7
3.1. Separación de responsabilidades	7
3.2. Control de versiones durante el desarrollo	7
4. Modelo relacional y conservación del EER	8
4.1. Diagrama Entidad-Relación	8
4.2. Importación y respaldo	8
5. MongoDB Atlas: modelo activo de reseñas	10
5.1. Reglas de negocio	10
5.2. Consistencia y manejo de respuestas	10
6. Funciones implementadas y flujo de compra	11
6.1. Catálogo, carrito y promociones	11
6.2. Checkout transaccional	11
6.3. Recuperación sin duplicados	11
6.4. Paneles, imágenes y países	11
7. API, autenticación y seguridad	12
7.1. Controles y respuestas	12
7.2. Protección de datos y acceso	12
8. Instalación y entornos	13

8.1. Ejecución local	13
8.2. Dependencias y compilación	13
8.3. Configuración privada	13
8.4. Sitio publicado	13
9. Pruebas y verificación funcional	14
9.1. Resultados comprobados	14
9.2. Recursos de prueba	14
9.3. Procesos cubiertos	14
10. Evidencias y matriz de cumplimiento	15
10.1. Recursos de la entrega	15
11. Presentación y conclusiones	16
11.1. Guion de demostración	16
11.2. Conclusiones	16
11.3. Fuentes internas	16

1. Resumen ejecutivo y planteamiento

1.1 Contexto y problema

Una tienda necesita reunir productos, existencias, clientes, pedidos y atención posterior. Administrar esos datos por separado dificulta mantener la coherencia entre lo que se ofrece, lo que compra un usuario y los documentos que respaldan la operación. Los procesos tradicionales de compra de productos pueden presentar limitaciones relacionadas con la disponibilidad de información, consulta de productos, gestión de pedidos y seguimiento de las compras en tiempo real. Al realizar estos procesos de forma manual o por medio de canales no integrados, el cliente puede tener dificultades para consultar productos, conocer sus características, administrar su compra y verificar el estado de sus pedidos.

1.2 Objetivo general

Desarrollar una aplicación web de comercio electrónico que permita administrar un catálogo de productos y ofrecer a los usuarios un proceso de compra sencillo, organizado y seguro que permita consultar productos, registrar clientes, autenticarse, gestionar un carrito de compra y crear pedidos mediante una interfaz responsiva conectada a servicios HTTP y almacenamiento persistente.

1.3 Solución implementada

React y Vite presentan el catálogo y los paneles. PHP verifica sesiones, permisos y reglas de negocio. MariaDB conserva usuarios, productos, pedidos, facturas, pagos y devoluciones. MongoDB Atlas almacena las reseñas activas. PHPMailer envía confirmaciones de prueba capturadas por Ethereum y una API externa de países que complementa las direcciones.

Elemento	Inventario comprobado
Esquema relacional	17 tablas; Resenas SQL se conserva como histórica.
Interfaz	5 páginas y 25 archivos JSX en components.
Backend	15 modelos y 16 controladores PHP.
API	18 subdirectorios de recursos.
Datos de apoyo	17 CSV y 17 JSON de exportación.
Servicios específicos	Correo, reseñas MongoDB e imágenes de productos/reseñas.

1.4 Alcance funcional

TodoAquí integra búsqueda de productos, registro y autenticación de usuarios, carrito de compra, panel personal, administración, reseñas con imágenes y solicitudes de devolución. El proceso de compra registra órdenes, pagos y facturas internas, y actualiza el inventario de forma transaccional. Estos componentes permiten desarrollar el ciclo comercial previsto para el proyecto académico.

La entrega incluye la aplicación funcional, el modelo de datos, las exportaciones SQL, CSV y JSON, las evidencias de funcionamiento y la documentación técnica.

2. Requerimientos y Product Backlog

Los requerimientos se han organizado utilizando un *Product Backlog*, que permite identificar las funcionalidades que desarrolla el sistema.

2.1 Requerimientos funcionales

ID	Función	Criterio de aceptación
RF01	Consultar productos	Nombre, categoría, precio, imagen, descripción y existencias.
RF02	Buscar y filtrar	Búsqueda, categorías/descendientes, novedades, vendidos y ofertas.
RF03	Vista rápida	Datos del producto y reseñas disponibles.
RF04	Autenticación	Registro, login/logout, hash y permisos en servidor.
RF05	Carrito	Agregar, ajustar, retirar y conservar cantidades en localStorage.
RF06	Favoritos	Lista personal persistida y sincronizada con el catálogo.
RF07	Compra simulada	Direcciones, nombre/NIT, método, documentos y stock transaccionales.
RF08	Recuperar confirmación	Mismo identificador sin duplicar la compra.
RF09	Correo de prueba	Envío después del commit; un fallo no cancela el pedido.
RF10	Datos personales	Perfil, contraseña y direcciones propias.
RF11	Consultar compras	Órdenes, detalles, facturas y pagos del propietario.
RF12	Reseñar compra entregada	Calificación 1–5, unicidad, comentario y hasta tres imágenes.
RF13	Solicitar devolución	Orden entregada, cantidades disponibles y estado.
RF14	Administrar catálogo	Productos, categorías, proveedores, promociones y relaciones.
RF15	Gestionar documentos	Recursos administrativos con reglas y transiciones.
RF16	Moderar reseñas	Ocultar/publicar sin reescribir contenido ajeno.
RF17	Cargar imágenes	Validación de contenido/tamaño y vista previa de producto.
RF18	Países y direcciones	Guatemala inicial, selección, conservación y reintento.
RF19	Interfaz móvil	Navegación y formularios adaptados.

2.2 Requerimientos no funcionales

ID	Objetivo	Mecanismo
RNF01	Acceso controlado	Sesiones, roles, propiedad y consultas preparadas.
RNF02	Integridad	FK/UNIQUE, transacciones, bloqueos SQL e índices MongoDB.
RNF03	Importes coherentes	BCMath y cálculo en servidor.
RNF04	Recuperación	Idempotencia de compra y tratamiento de errores externos.
RNF05	Usabilidad	Español, validaciones, estados vacíos/errores y diseño responsivo.
RNF06	Rendimiento	Compilación Vite, caché y peticiones compartidas.
RNF07	Mantenibilidad	Capas, lockfiles y documentación.
RNF08	Privacidad	Configuración privada y respuestas sin contraseñas.

2.3 Validación de requerimientos

La verificación del proyecto confirmó el cumplimiento de los aspectos y requisitos mínimos establecidos para la entrega académica. Los requerimientos funcionales y no funcionales se concretan en la interfaz, los servicios HTTP, la persistencia de datos y las integraciones utilizadas por la aplicación.

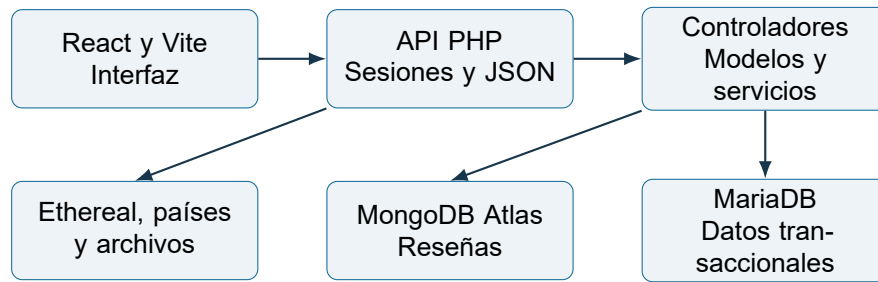
El proceso de compra, la conexión con MongoDB Atlas, el envío de confirmaciones mediante Ethereum y la publicación del sitio con certificado SSL validado forman parte de los resultados comprobados.

2.4 Mockups, referencias visuales y versión de interfaz

La interfaz toma como referencia la estructura general de un ecommerce moderno: encabezado, búsqueda, categorías, hero, tarjetas de producto, promociones, carrito lateral y pie de página. La implementación se adapta a TodoAquí mediante componentes React y estilos propios.

Para la presentación se recomienda mostrar: inicio en escritorio, menú hamburguesa en móvil, catálogo con filtros, vista rápida, carrito, login y registro, checkout y confirmación de compra.

3. Arquitectura y tecnologías



Capa	Tecnologías y función
Frontend	React 19, React Router 7, Vite 8, Bootstrap 5.3, SCSS y JavaScript.
Backend	PHP procedural, endpoints independientes, controladores/modelos y servicios.
Datos	PDO/MariaDB; extensión PHP mongodb y biblioteca Composer para Atlas.
Correo	PHPMailer mediante SMTP; Ethereum en la instalación local.
Países	cURL hacia REST Countries; caché temporal y control de concurrencia.
Pruebas	Python para HTTP/SQL; Node/Playwright para interfaz; mongod privado para reseñas.

3.1 Separación de responsabilidades

`api/` gestiona HTTP, sesión y permisos; `app/controllers/` valida reglas; `app/models/` consulta datos y `app/services/` trata correo, imágenes y reseñas. La configuración está en `config/`; datos y scripts en `database/`; guías en `docs/`.

En React, *pages* contiene pantallas; *components* controles y vistas; *context* estado/hooks; *services* clientes HTTP; *data* esquemas de paneles; *styles* y *js* estilos/interacciones. *vendor* y *node_modules* son dependencias, no código propio para personalizar manualmente.

3.2 Control de versiones durante el desarrollo

Durante el desarrollo se utilizó un repositorio Git para registrar los cambios del código fuente, organizar las versiones y apoyar el trabajo colaborativo. El control de versiones formó parte del proceso de construcción y evolución del proyecto.

4. Modelo relacional y conservación del EER

MariaDB conserva la operación transaccional. El esquema reproducible está en `database/crear_bd_tablas.sql`; el dump `database/todoaqui_db.sql` contiene la estructura y los datos exportados para reproducir la base relacional.

Dominio	Tablas
Clientes	Usuarios, Direcciones y ListaDeseos.
Catálogo	Categorías, Productos, Proveedores y ProductosProveedores.
Promociones	Promociones y ProductosPromociones.
Ventas	Ordenes, DetalleOrdenes, Pagos, Facturas y DetalleFacturas.
Posventa	Devoluciones y DetalleDevoluciones.
Histórico	Resenas SQL; las reseñas activas se gestionan en MongoDB.

4.1 Diagrama Entidad-Relación

La página siguiente presenta el diagrama EER, con sus entidades, atributos, claves y relaciones. El modelo organiza los dominios de clientes, catálogo, ventas y posventa, y permite comprender la integridad referencial de la operación comercial.

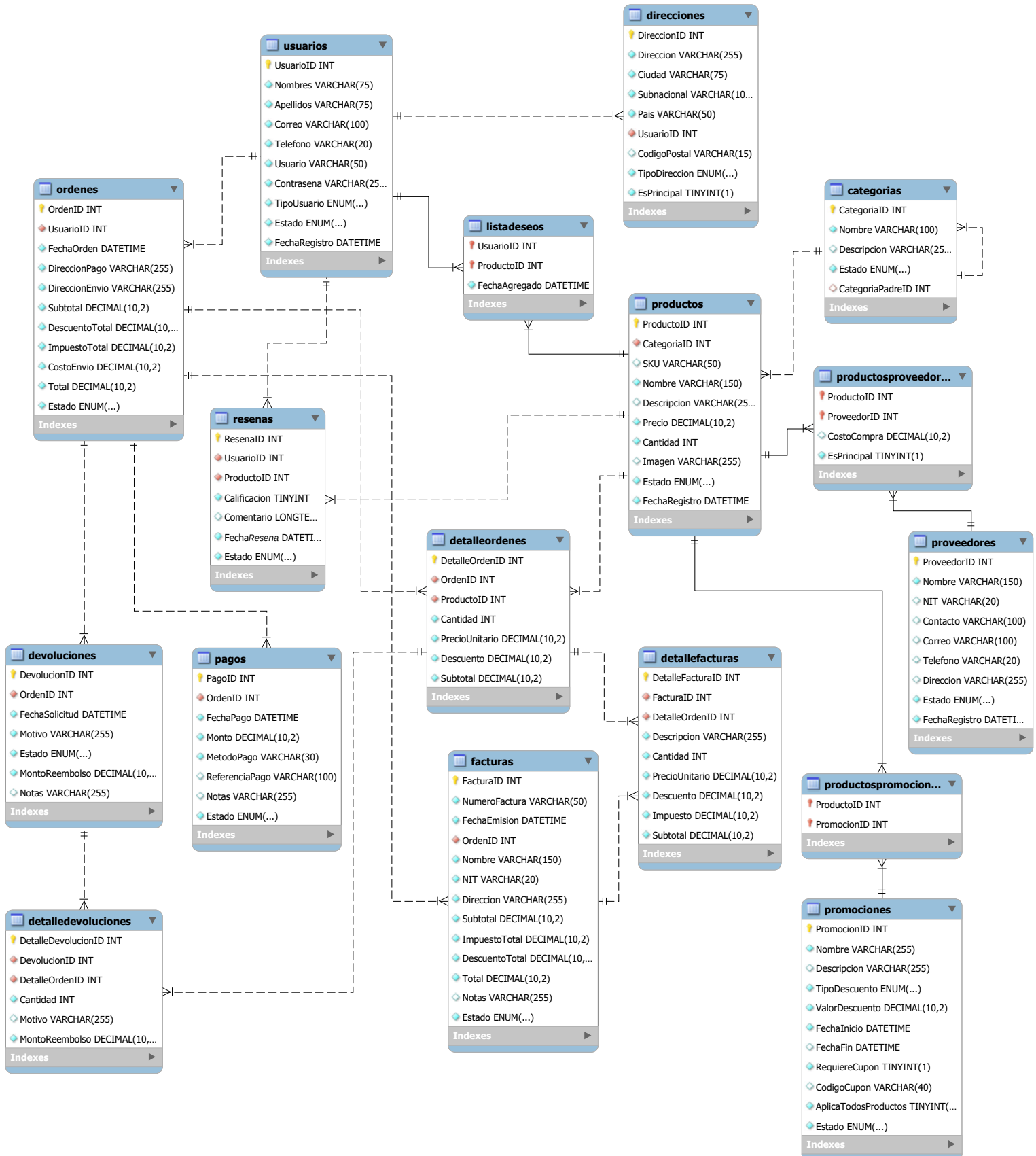
La tabla relacional de reseñas forma parte del modelo histórico; la gestión activa de opiniones se realiza en MongoDB Atlas. El esquema SQL documenta los nombres y las restricciones utilizados por la aplicación. La versión cargada en el servidor hosting utiliza el modelo histórico mientras la versión local se mantiene con MongoDB Atlas.

4.2 Importación y respaldo

La preparación de la base relacional dispone de dos alternativas: importar el archivo SQL con estructura y datos, o crear el esquema y cargar los archivos CSV. Las exportaciones incluyen 17 CSV y 17 JSON, que complementan los recursos de datos del proyecto.

El respaldo del sistema comprende la información relacional de MariaDB, las colecciones de MongoDB y las fotografías almacenadas en disco. Esta organización conserva tanto los registros de negocio como los archivos asociados.

Diagrama Entidad-Relación (EER)



5. MongoDB Atlas: modelo activo de reseñas

MongoDB Atlas está integrado y su conexión desde el sitio local fue verificada. La aplicación crea, consulta, modifica, modera y elimina reseñas en la colección `Resenas`; `Contadores` asigna identificadores numéricos.

Campo	Finalidad
ResenaID	Identificador numérico compatible con API e interfaz.
UsuarioID / ProductoID	Referencias lógicas a registros de MariaDB.
Calificacion / Comentario	Valor entero 1–5 y texto de la opinión.
FechaResena / Estado	Fecha BSON y visibilidad Publicada/Oculto.
Imagenes	Arreglo de rutas de fotografías, no binarios base64.
Version	Control de concurrencia en edición de contenido.

5.1 Reglas de negocio

- Compra entregada del producto para publicar y unicidad usuario/producto.
- El propietario edita su contenido; el administrador modera sin reescribir opiniones ajenas.
- Las ocultas no aparecen en listados públicos ni promedios.
- Máximo tres fotografías por reseña, de un máximo de 2 MB cada una; JPG, PNG o WebP.
- Las fotos se guardan en `public/uploads/resenas`; MongoDB conserva sus rutas.

5.2 Consistencia y manejo de respuestas

La aplicación valida las referencias de usuarios y productos entre MariaDB y MongoDB. El control de versiones de las reseñas detecta conflictos de edición concurrente y responde con el código HTTP 409. El manejo de disponibilidad de servicios utiliza respuestas específicas y mensajes en la interfaz, permitiendo conservar la navegación del catálogo.

6. Funciones implementadas y flujo de compra

6.1 Catálogo, carrito y promociones

El catálogo utiliza productos/categorías activos y ofrece búsqueda, categorías descendientes, novedades, vendidos y ofertas. Aplica la mejor promoción automática elegible sin acumular descuentos; no aplica cupones como ofertas automáticas. El carrito conserva ID/cantidad en localStorage y recupera datos/precios del catálogo.

6.2 Checkout transaccional

1. El usuario inicia sesión, revisa cantidades, selecciona direcciones propias o introduce otras y proporciona nombre/NIT y método de pago.
2. El servidor valida propietario, productos, stock y precios esperados; los importes enviados por el navegador no son autoridad.
3. Una transacción guarda Ordenes, DetalleOrdenes, Facturas, DetalleFacturas y Pagos, y descuenta stock.
4. Se confirma con commit o se revierte el conjunto ante error. El pedido y pago comienzan pendientes.
5. Después del commit se intenta el correo. Su fallo no anula la compra ni requiere comprar otra vez.

6.3 Recuperación sin duplicados

SolicitudID identifica un intento de compra y se conserva por usuario en sessionStorage. Repetir identificador y datos devuelve la orden existente; reutilizarlo con otros datos genera conflicto. La confirmación pertenece al usuario autenticado y recuperar el intento no reenvía automáticamente el correo.

6.4 Paneles, imágenes y países

El panel personal incluye perfil, direcciones, órdenes, deseos, reseñas y devoluciones. Administración gestiona 17 recursos con restricciones: las órdenes cambian de estado, las facturas se anulan y los detalles documentales están sujetos a reglas de integridad.

Productos admiten fotografías con validación de contenido, tamaño y dimensiones. Países se obtienen mediante PHP desde REST Countries, con caché de 24 horas, bloqueo y reintento. Las direcciones manuales separan dirección, ciudad, departamento y país; las guardadas del checkout se muestran de solo lectura.

Los métodos de efectivo y transferencia generan registros de pago con estado inicial pendiente. Las facturas internas documentan las operaciones de compra dentro del alcance académico del proyecto.

7. API, autenticación y seguridad

La base local es `/proyecto-desarrollador-full-stack/api/`. Los recursos utilizan endpoints `index.php` y la autenticación dispone de archivos específicos para cada operación.

Grupo	Contrato general
productos / categorías	Consulta pública; escritura administrativa.
auth	POST de registro/login/logout y GET de sesión.
usuarios / proveedores / promociones	Gestión administrativa.
catalogo / checkout	GET público de catálogo; compra/confirmación autenticadas.
cuenta / wishlist	Datos y favoritos del propietario.
ordenes / pagos / facturas / devoluciones	Acceso por propietario/rol y transiciones permitidas.
resenas	Consulta pública filtrada; autoría y moderación.
admin	Relaciones y detalles con claves simples/compuestas.
países	Listado GET y caché del proveedor externo.

7.1 Controles y respuestas

Se utilizan `password_hash/password_verify`, consultas preparadas, verificación de usuario activo y permisos en PHP. Las validaciones cubren IDs, textos, montos, fechas y contenido de imágenes. Las operaciones JSON requieren el tipo adecuado; productos/reseñas aceptan multipart para archivos.

Los códigos principales son 200/201 (éxito), 400 (validación), 401/403 (acceso), 404 (registro inexistente o ajeno), 405 (método), 409 (conflicto), 415 (contenido), 500 (error interno) y 503 (servicio no disponible en los casos previstos).

7.2 Protección de datos y acceso

Las contraseñas se almacenan mediante hash y se verifican en el servidor. Las sesiones y los permisos delimitan las operaciones del cliente y del administrador. La validación de propiedad protege la consulta y modificación de los datos personales y documentos de cada usuario.

La configuración de los servicios se mantiene separada de las respuestas públicas. Las consultas preparadas y las validaciones del backend refuerzan el tratamiento de los datos recibidos.

8. Instalación y entornos

8.1 Ejecución local

La carpeta vigente es `C:/xampp/htdocs/proyecto-desarrollador-full-stack`. Iniciar Apache y MariaDB, comprobar configuración y dependencias, y ejecutar desde la raíz:

```
npm.cmd run dev
```

Vite utiliza inicialmente el puerto 5173. Detecta Apache en `localhost:80` y después `localhost:8080` consultando categorías. Su proxy dirige `/proyecto-desarrollador-full-stack/api` y las fotografías de reseñas al backend detectado. Usar la URL que informe Vite.

8.2 Dependencias y compilación

Composer requiere PHP y extensiones compatibles: el lockfile de MongoDB exige `ext-mongodb` de la serie 2.4 compatible. Se utilizan PDO MySQL, BCMath, cURL, OpenSSL y fileinfo. El Vite instalado declara Node compatible con 20.19 o la rama admitida desde 22.12.

En una copia nueva, instalar las dependencias a partir de los lockfiles:

```
composer install
npm.cmd --prefix frontend ci
```

Para comprobaciones y compilación: `npm.cmdrunlint` y `npm.cmdrunbuild`. La salida es `frontend/dist`, distinta del directorio raíz `build`.

8.3 Configuración privada

MariaDB usa `config/database.php`. MongoDB admite `MONGODB_URI` y `MONGODB_DATABASE`, con URI alternativa privada en `api_keys.php`. El correo lee variables SMTP y después `mail.local.php`, que tiene precedencia. La configuración privada reúne los parámetros necesarios para conectar los servicios utilizados por la aplicación.

8.4 Sitio publicado

<https://proyecto6-dfs-intecap2026.x10.mx/>

El sitio se encuentra publicado y cuenta con un certificado SSL validado, que habilita el acceso mediante HTTPS. El directorio `build` reúne los archivos cargados al hosting.

La entrega incluye una copia completa del directorio del proyecto funcionando localmente. En este entorno se verificaron la conexión a MongoDB Atlas, la creación de una nueva orden y el envío de su correo de confirmación mediante Ethereum.

9. Pruebas y verificación funcional

9.1 Resultados comprobados

El proyecto cumple los aspectos y requisitos mínimos de la entrega académica. La verificación funcional abarca los procesos de consulta, gestión y compra, junto con la persistencia de datos y las integraciones de la aplicación.

La conexión local a MongoDB Atlas fue comprobada. También se realizó la creación de una nueva orden y se verificó el envío de su correo de confirmación mediante Ethereum. El sitio publicado dispone de un certificado SSL validado.

9.2 Recursos de prueba

El proyecto incluye rutinas automatizadas y casos de comprobación para apoyar la validación y el mantenimiento de sus funcionalidades.

Rutina	Cobertura
tests/test_api.py	Operaciones HTTP y SQL, validaciones y reglas de acceso por módulo.
tests/test_resenas_mongo.py	Gestión de reseñas y reglas de integración entre MariaDB y MongoDB en un entorno de prueba.
tests/test_imagenes_productos.py	Carga y validación de fotografías de productos.
tests/*_ui.cjs	Interacciones de paneles y checkout mediante escenarios de interfaz.
npm.cmdrunlint/build	Análisis del código frontend y generación de los archivos compilados.

9.3 Procesos cubiertos

- Consulta del catálogo, búsqueda, filtros y vista rápida en una interfaz responsiva.
- Registro, inicio de sesión y acceso a los datos del usuario.
- Administración del carrito, creación de órdenes y actualización de existencias.
- Recuperación de la confirmación de compra mediante el identificador de solicitud.
- Gestión de reseñas, fotografías y moderación de su visibilidad.
- Solicitudes de devolución y gestión administrativa de documentos.
- Consulta de países e integración del correo de confirmación.

10. Evidencias y matriz de cumplimiento

10.1 Recursos de la entrega

El directorio INFORME reúne el informe del proyecto, el diagrama EER en PDF, el modelo editable de MySQL Workbench y el archivo ZIP de entrega. Incluye evidencias de la integración con MongoDB, el correo de confirmación, la consulta de países y el certificado SSL del sitio publicado.

La documentación técnica describe la instalación, la arquitectura, los modelos de datos, la API y los flujos de uso. Estos recursos acompañan la aplicación y respaldan su presentación.

Aspecto	Implementación o evidencia	Estado
Problema y objetivos	Planteamiento y objetivos del proyecto.	Cumplido.
Requerimientos	19 requerimientos funcionales y 8 no funcionales organizados en el backlog.	Cumplido.
Modelo relacional	Esquema SQL, diagrama EER y modelo Workbench.	Cumplido.
Modelo NoSQL	Reseñas integradas con MongoDB Atlas y conexión local verificada.	Cumplido.
Arquitectura	Tecnologías y responsabilidades organizadas por capas.	Cumplido.
CRUD y API	Endpoints, controladores, modelos y paneles de gestión.	Cumplido.
Compra y correo	Creación de orden y envío de confirmación mediante Ethereum verificados.	Cumplido.
Publicación HTTPS	Sitio publicado con certificado SSL validado.	Cumplido.
Verificación	Comprobación de los aspectos y requisitos mínimos del proyecto.	Cumplido.
Interfaz	Catálogo, formularios y paneles responsivos.	Cumplido.
Documentación y entrega	Guías técnicas, informe y copia del proyecto en archivo ZIP.	Cumplido.
Control de versiones	Repositorio Git utilizado durante el desarrollo.	Cumplido.

11. Presentación y conclusiones

11.1 Guion de demostración

La presentación inicia con el problema, los objetivos, los requerimientos y la arquitectura. Continúa con el catálogo, los filtros, el carrito y el inicio de sesión. El proceso de compra muestra la confirmación, la persistencia de los documentos y la actualización del inventario.

Los paneles personal y administrativo permiten mostrar la gestión de información, las reseñas y las imágenes. La exposición se completa con el diagrama EER, la distribución de responsabilidades entre MariaDB y MongoDB, el correo de confirmación y el acceso HTTPS al sitio publicado.

11.2 Conclusiones

TodoAquí integra una interfaz responsiva con servicios PHP, almacenamiento relacional en MariaDB y reseñas en MongoDB Atlas. El sistema reúne el catálogo, la administración de usuarios, el carrito, las órdenes, los pagos internos, las facturas y la atención posterior a la compra.

El checkout coordina los documentos de venta y el inventario mediante transacciones. La recuperación de solicitudes evita duplicar compras y el envío de correo acompaña la confirmación de la orden. Los paneles permiten gestionar los datos según el rol y la propiedad de los registros.

La conexión a MongoDB Atlas, la creación de órdenes y el envío de confirmaciones mediante Ethereum se comprobaron en el entorno local. La publicación con SSL validado complementa la entrega de la aplicación. El uso de Git durante el desarrollo, la documentación y los modelos de datos respaldan el proceso realizado.

El proyecto cumple los aspectos y requisitos mínimos establecidos para la entrega académica y presenta una solución funcional para los procesos de comercio electrónico definidos en su alcance.

11.3 Fuentes internas

- Documentación en docs: instalación, arquitectura, datos, backlog, API, checkout, paneles, reseñas, imágenes, países y pruebas.
- Código del proyecto, manifiestos de dependencias y esquema SQL.
- Informe del 25/09/2026, Diagrama_EER.pdf y Modelo_SQL.mwb.
- Resultados de verificación funcional e integración y evidencias incluidas en la entrega.